

Project Report: Anime Recommendation System

Hediyeh Nayebi

July 6, 2026

1 Introduction

In the contemporary digital era, the proliferation of online content has necessitated the development of sophisticated recommendation systems to assist users in navigating vast information spaces. Among these, the anime domain presents a unique challenge due to the diverse preferences of the user base and the hierarchical nature of anime metadata.

This project aims to design and implement a robust recommendation engine tailored for the anime dataset. The core objective is to move beyond simple popularity-based suggestions by leveraging Collaborative Filtering (CF) techniques. By analyzing user-item interaction patterns—specifically rating behaviors—we aim to uncover latent features that drive user preferences.

The primary contributions of this report are threefold:

1. **Data Exploration:** A comprehensive analysis of user-item interaction matrices to understand sparsity and interaction distributions.
2. **Noise Reduction:** Implementation of data preprocessing techniques to mitigate the "cold-start" problem and improve dataset quality.
3. **Model Evaluation:** Development and assessment of recommendation algorithms, evaluated against standard metrics such as Root Mean Square Error (RMSE) to ensure prediction accuracy.

Through this systematic approach, we intend to provide a scalable and effective solution capable of delivering personalized recommendations within the high-dimensional anime ecosystem.

2 Data Exploration and Matrix Analysis

To understand the dataset structure, we performed an initial analysis of the user-item interaction matrix. The following Python code was used to calculate the matrix density and sparsity:

Listing 1: Calculating Matrix Sparsity

```
1      # Calculate dimensions
2      num_users = rating_df['user_id'].nunique()
3      num_animes = rating_df['anime_id'].nunique()
4      total_ratings = len(rating_df)
5
6      # Sparsity calculation
7      density = total_ratings / (num_users * num_animes)
8      sparsity = 1 - density
9
10     print(f"Matrix Sparsity: {sparsity*100:.2f}%")
```

2.1 Visualizing the Long Tail

We visualized the interaction frequency to identify the "Long Tail" phenomenon, where a small number of anime titles receive a disproportionate number of ratings.

Listing 2: Plotting Interaction Frequency

```
1      # Plotting the Long Tail distribution
2      anime_popularity = rating_df.groupby('anime_id')['
      ↪ rating'].count()
3      plt.plot(np.sort(anime_popularity.values))
4      plt.yscale('log') # Log scale to handle the skewness
5      plt.title('Anime Interaction Frequency (Long Tail)')
6      plt.show()
```

2.2 Mathematical Context

The interaction dataset is modeled as a matrix $R \in \mathbb{R}^{U \times I}$, where U denotes the set of users, I denotes the set of anime titles, and r_{ui} represents the rating given by user u to anime i .

Given that most users interact with only a tiny fraction of the available anime, R is extremely sparse. The sparsity is mathematically defined as:

$$\text{Sparsity} = 1 - \frac{|\{r_{ui} \mid r_{ui} \neq 0\}|}{|U| \times |I|} \quad (1)$$

Furthermore, the interaction frequency visualized in the long-tail plot follows a power-law distribution. This is expressed as $P(x) \propto x^{-\alpha}$, indicating that a highly skewed minority of items receives the vast majority of interactions.

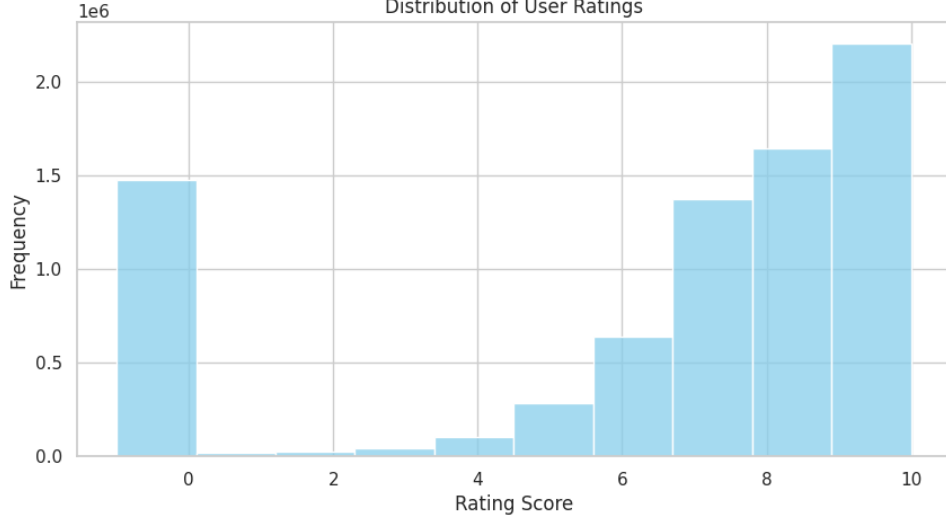


Figure 1: Distribution of User Ratings

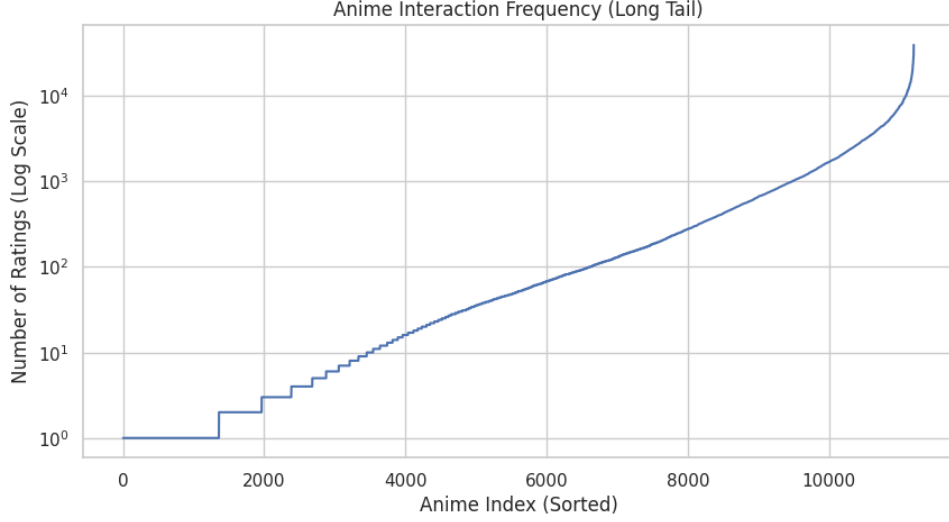


Figure 2: Long Tail Frequency (Logarithmic Scale)

3 Data Preprocessing & Noise Reduction

To address the cold-start problem and mitigate the computational challenges imposed by the high sparsity of the user-item interaction matrix, a robust data filtering pipeline was implemented. In collaborative filtering systems, users with very few interactions often introduce high noise and minimal signal, which severely limits the model’s ability to learn stable and reliable preference profiles.

Therefore, an activity threshold τ is defined such that any user u with a total number of interactions less than τ is excluded from the active dataset. Formally, let $R_u = \{i \in I \mid r_{ui} \neq 0\}$ represent the set of items rated by user u . The filtering criterion for the active user base is expressed as:

$$U_{\text{filtered}} = \{u \in U \mid |R_u| \geq \tau\}$$

For this implementation, the threshold was set to $\tau = 50$. This preprocessing step ensures that the recommendation engine trains exclusively on high-quality user profiles with well-established historical preferences, thereby significantly reducing matrix sparsity and optimizing computational efficiency.

The quantitative impact of this filtering process on the dataset dimensions is summarized in Table 1.

Table 1: Dataset Statistics Before and After Noise Reduction ($\tau = 50$)

Metric	Original Dataset	Filtered Dataset	Impact (% Change)
Total Ratings	7,813,737	7,166,253	-8.29%
Unique Users	73,515	39,466	-46.32%

4 User-Item Matrix Construction

Following the noise reduction phase, the longitudinal dataset was transformed into a wide-format user-item interaction matrix $R \in \mathbb{R}^{U \times I}$, where rows correspond to unique users and columns to anime titles. Each element r_{ui} represents the rating given by user u to anime i .

In cases of duplicated interactions (e.g., a user rating the same anime multiple times due to data logging anomalies), the arithmetic mean of the duplicate ratings was computed to aggregate the preferences into a single representative value. Finally, all unobserved interactions were explicitly imputed with a value of 0. This structural transformation is a fundamental prerequisite for computing vector-based similarities in collaborative filtering architectures.

5 Similarity Computation

To quantify the relationships between different anime titles based on user preferences, an Item-Based Collaborative Filtering approach was adopted. This method is computationally more stable and efficient than User-Based approaches due to the static nature of item attributes compared to dynamic user behaviors.

The similarity between anime vectors was measured using Cosine Similarity. For two anime items i and j , represented as vectors $\mathbf{v}_i$ and $\mathbf{v}_j$ in the user-interaction space, the similarity score $sim(i, j)$ is mathematically defined as the cosine of the angle between them:

$$sim(i, j) = \frac{\mathbf{v}_i \cdot \mathbf{v}_j}{\|\mathbf{v}_i\| \|\mathbf{v}_j\|} \quad (2)$$

This operation yielded a symmetric, square similarity matrix $S \in \mathbb{R}^{I \times I}$, where I represents the total number of unique anime titles (11,162). Each cell $s_{ij} \in [0, 1]$ indicates the strength of the collaborative relationship between item i and item j , forming the mathematical foundation for the recommendation engine.

6 Recommendation Generation

Once the symmetric item similarity matrix $S \in \mathbb{R}^{I \times I}$ is constructed, it serves as the core look-up database for the recommendation engine. The generation of recommendation lists follows an Item-Based Collaborative Filtering heuristic.

Given a target anime i , the system queries the corresponding row or column in the similarity matrix to retrieve the similarity scores s_{ij} for all other anime titles $j \in I$ where $j \neq i$. The scores are then sorted in descending order:

$$\text{Sorted_Similarities}(i) = \text{sort_descending}(\{s_{ij} \mid j \in I \setminus \{i\}\}) \quad (3)$$

The top N items with the highest cosine similarity scores are extracted to form the final recommendation vector:

$$\mathbf{R}_N(i) = \{j_1, j_2, \dots, j_N\} \quad (4)$$

This approach ensures that the recommended anime titles structurally align with the historical preference patterns of users who interacted with the target item, thereby providing contextually relevant and behaviorally grounded recommendations.

7 Metadata Integration and Final Output

While the collaborative filtering engine successfully computes vector distances to identify similar items, the raw output consists of numerical item identifiers (`anime_id`). To enhance interpretability and user experience, a secondary metadata dataset (`anime.csv`) was integrated.

A mapping function was implemented to cross-reference the recommended IDs with this metadata repository. By performing a relational merge operation (left join), the system translates the raw ID vectors into human-readable anime titles. The final pipeline seamlessly takes a target anime, computes the top- N similar items via the cosine similarity matrix, and outputs a structured table containing the actual anime names alongside their respective confidence scores. This transformation is a crucial step for practical deployment in a production environment.